Add:

```
@theg2cuber ➜/workspaces/theg2cuber/module5 (main) $ nasm -f elf32 LabExer1.asm -o LabExer1.o
@theg2cuber ➜/workspaces/theg2cuber/module5 (main) $ ld -m elf_i386 LabExer1.o -o LabExer1
@theg2cuber ➜/workspaces/theg2cuber/module5 (main) $ ./LabExer1
----------------------------
        Menu
[A]dd
[S]ubtract
[M]ultiply
[D]ivide
----------------------------
Enter your Choice: A
Enter First number: 2
Enter Second number: 3
The sum is 5
Do you want to try again? (Y/N):
```

```
@theg2cuber ➜/workspaces/theg2cuber/module5 (main) $ ./LabExer1
        Menu
[A]dd
[S]ubtract
[M]ultiply
[D]ivide
----------------------------
Enter your Choice: A
Enter First number: 42
Enter Second number: 57
The sum is 99
Do you want to try again? (Y/N):
```

```
@theg2cuber ➜/workspaces/theg2cuber/module5 (main) $ ./LabExer1
        Menu
[A]dd
[S]ubtract
[M]ultiply
[D]ivide
----------------------------
Enter your Choice: a
Enter First number: 9
Enter Second number: 99
The sum is 108
Do you want to try again? (Y/N):
```

Subtraction:

```
@theg2cuber →/workspaces/theg2cuber/module5 (main) $ ./LabExer1
        Menu
[A]dd
[S]ubtract
[M]ultiply
[D]ivide
----------------------------
Enter your Choice: S
Enter First number: 80
Enter Second number: 50
The difference is 30
Do you want to try again? (Y/N):
```

```
@theg2cuber →/workspaces/theg2cuber/module5 (main) $ ./LabExer1
        Menu
[A]dd
[S]ubtract
[M]ultiply
[D]ivide
----------------------------
Enter your Choice: s
Enter First number: 67
Enter Second number: 7
The difference is 60
Do you want to try again? (Y/N):
```

```
@theg2cuber →/workspaces/theg2cuber/module5 (main) $ ./LabExer1
        Menu
[A]dd
[S]ubtract
[M]ultiply
[D]ivide
----------------------------
Enter your Choice: s
Enter First number: 9
Enter Second number: 3
The difference is 6
Do you want to try again? (Y/N):
```

Multiply:

```
@theg2cuber →/workspaces/theg2cuber/module5 (main) $ ./LabExer1
        Menu
[A]dd
[S]ubtract
[M]ultiply
[D]ivide
----------------------------
Enter your Choice: m
Enter First number: 6
Enter Second number: 7
The product is 42
Do you want to try again? (Y/N):
```

```
@theg2cuber →/workspaces/theg2cuber/module5 (main) $ ./LabExer1
        Menu
[A]dd
[S]ubtract
[M]ultiply
[D]ivide
----------------------------
Enter your Choice: M
Enter First number: 16
Enter Second number: 3
The product is 48
Do you want to try again? (Y/N):
```

```
@theg2cuber →/workspaces/theg2cuber/module5 (main) $ nasm -f elf32 LabExer1.asm -o LabExer1.o
@theg2cuber →/workspaces/theg2cuber/module5 (main) $ ld -m elf_i386 LabExer1.o -o LabExer1
@theg2cuber →/workspaces/theg2cuber/module5 (main) $ ./LabExer1
----------------------------
        Menu
[A]dd
[S]ubtract
[M]ultiply
[D]ivide
----------------------------
Enter your Choice: m
Enter First number: 32
Enter Second number: 10
The product is 320
Do you want to try again? (Y/N):
```

Division:

```
@theg2cuber ➜/workspaces/theg2cuber/module5 (main) $ nasm -f elf32 LabExer1.asm -o LabExer1.o
@theg2cuber ➜/workspaces/theg2cuber/module5 (main) $ ld -m elf_i386 LabExer1.o -o LabExer1
@theg2cuber ➜/workspaces/theg2cuber/module5 (main) $ ./LabExer1
----------------------------
        Menu
[A]dd
[S]ubtract
[M]ultiply
[D]ivide
----------------------------
Enter your Choice: d
Enter First number: 10
Enter Second number: 5
The quotient is 2
Do you want to try again? (Y/N):
```

```
@theg2cuber ➜/workspaces/theg2cuber/module5 (main) $ nasm -f elf32 LabExer1.asm -o LabExer1.o
@theg2cuber ➜/workspaces/theg2cuber/module5 (main) $ ld -m elf_i386 LabExer1.o -o LabExer1
@theg2cuber ➜/workspaces/theg2cuber/module5 (main) $ ./LabExer1
----------------------------
        Menu
[A]dd
[S]ubtract
[M]ultiply
[D]ivide
----------------------------
Enter your Choice: d
Enter First number: 70
Enter Second number: 3
The quotient is 23
Do you want to try again? (Y/N):
```

```
@theg2cuber ➜/workspaces/theg2cuber/module5 (main) $ nasm -f elf32 LabExer1.asm -o LabExer1.o
@theg2cuber ➜/workspaces/theg2cuber/module5 (main) $ ld -m elf_i386 LabExer1.o -o LabExer1
@theg2cuber ➜/workspaces/theg2cuber/module5 (main) $ ./LabExer1
----------------------------
        Menu
[A]dd
[S]ubtract
[M]ultiply
[D]ivide
----------------------------
Enter your Choice: d
Enter First number: 36
Enter Second number: 9
The quotient is 4
Do you want to try again? (Y/N):
```

Loop:

```
@theg2cuber →/workspaces/theg2cuber/module5 (main) $ nasm -f elf32 LabExer1.asm -o LabExer1.o
@theg2cuber →/workspaces/theg2cuber/module5 (main) $ ld -m elf_i386 LabExer1.o -o LabExer1
@theg2cuber →/workspaces/theg2cuber/module5 (main) $ ./LabExer1
----------------------------
        Menu
[A]dd
[S]ubtract
[M]ultiply
[D]ivide
----------------------------
Enter your Choice: d
Enter First number: 36
Enter Second number: 9
The quotient is 4
Do you want to try again? (Y/N): Y
----------------------------
        Menu
[A]dd
[S]ubtract
[M]ultiply
[D]ivide
----------------------------
Enter your Choice: 60
Invalid choice! Please select A, S, M, or D.
Do you want to try again? (Y/N): Y
----------------------------
        Menu
[A]dd
[S]ubtract
[M]ultiply
[D]ivide
----------------------------
Enter your Choice: A
Enter First number: 60
Enter Second number: 7
The sum is 67
Do you want to try again? (Y/N):
```

Explanation:

Right after the program starts, it clears the screen before doing anything, then goes inside the void_main to start the loop. Afterwards it displays the screen with show_Menu, get the choice input from the user and the numbers they want to use for the operation. Then it checks one by one if the choice is either a lowercase/uppercase A for addition, lowercase/uppercase S for subtraction, and so on for multiplication and division. Before it does the operation, it will first convert the user's number inputs into integers instead of ascii characters to get the actual arithmetic values. Once it sees a valid operation, since it already has the numbers the program wants to operate with, it can already start doing the operation. For example if the user enters: 'A', then '1', then '2', then the program jumps to do_Add, , moves the first value to eax, then adds the second value and prints the result. After all of that, it then prompts the user if they want

to do another operation and if they want to, then it repeats the process again, and if not, then it exits the program

Code:

```
;--- macro to print data easier ---
%macro printf 2
   mov eax, 4
   mov ebx, 1
   mov ecx, %1                          ;First input holds the variable to print
   mov edx, %2                          ;Second input holds the length of the variable
   int 0x80
%endmacro

;--- macro to get input from user easier ---
%macro scanf 2
   mov eax, 3
   mov ebx, 0
   mov ecx, %1                          ;Variable to save user input into
   mov edx, %2                          ;Value of reserved data for variable
   int 0x80
   cmp eax, 0
   jle %%done
   mov esi, %1
   mov byte [esi + eax - 1], 0
%%done:
%endmacro

;--- Macro to exit program ---
%macro Exit 0
   mov eax, 1
   mov ebx, 0
   int 0x80
%endmacro

section .data
   bar db '---------------------------- ', 10
   bar_Len equ $ - bar

   menu db 9, 'Menu', 10
   menu_Len equ $ - menu

   addition db '[A]dd', 10
   addition_Len equ $ - addition
```

```
subtraction db '[S]ubtract', 10
subtraction_Len equ $ - subtraction

multiply db '[M]ultiply', 10
multiply_Len equ $ - multiply

divide db '[D]ivide', 10
divide_Len equ $ - divide

enterChoice db 'Enter your Choice: '
enterChoice_Len equ $ - enterChoice

enterNum1 db 'Enter First number: '
enterNum1_Len equ $ - enterNum1

enterNum2 db 'Enter Second number: '
enterNum2_Len equ $ - enterNum2

resultSum db 'The sum is '
resultSum_Len equ $ - resultSum

resultDifference db 'The difference is '
resultDifference_Len equ $ - resultDifference

resultProduct db 'The product is '
resultProduct_Len equ $ - resultProduct

resultQuotient db 'The quotient is '
resultQuotient_Len equ $ - resultQuotient

divError db 'Cannot divide by zero!', 10
divError_Len equ $ - divError

invalidChoiceMsg db 'Invalid choice! Please select A, S, M, or D.', 10
invalidChoiceMsg_Len equ $ - invalidChoiceMsg

newLine db 10
newLine_Len equ 1

tryAgain db 'Do you want to try again? (Y/N): '
tryAgain_Len equ $ - tryAgain

clearScreen db 27, '[H', 27, '[2J'
clearScreen_Len equ $ - clearScreen
```

```
section .bss
    Choice resb 10
    Num1 resb 10
    Num2 resb 10
    Answer resb 12
    Value1 resd 1
    Value2 resd 1

section .text
    global _start

_start:
    ; Clear screen on initial startup
    printf clearScreen, clearScreen_Len

void_main:
    call show_Menu

    ; Validate Choice before asking for numbers
    mov al, [Choice]
    cmp al, 'A'
    je valid_choice
    cmp al, 'a'
    je valid_choice
    cmp al, 'S'
    je valid_choice
    cmp al, 's'
    je valid_choice
    cmp al, 'M'
    je valid_choice
    cmp al, 'm'
    je valid_choice
    cmp al, 'D'
    je valid_choice
    cmp al, 'd'
    je valid_choice

; Handle invalid choice
    printf invalidChoiceMsg, invalidChoiceMsg_Len
    jmp try_Again

valid_choice:
    ; 2. Get Numbers
```

```
    printf enterNum1, enterNum1_Len
    scanf Num1, 10
    mov esi, Num1
    call to_Int
    mov [Value1], eax

    printf enterNum2, enterNum2_Len
    scanf Num2, 10
    mov esi, Num2
    call to_Int
    mov [Value2], eax

    ; 3. Compute Result
    call compare_Choice
    jmp try_Again

show_Menu:
    printf bar, bar_Len
    printf menu, menu_Len
    printf addition, addition_Len
    printf subtraction, subtraction_Len
    printf multiply, multiply_Len
    printf divide, divide_Len
    printf bar, bar_Len

    printf enterChoice, enterChoice_Len
    scanf Choice, 10
    ret

compare_Choice:
    mov al, [Choice]

    cmp al, 'A'
    je do_Add
    cmp al, 'a'
    je do_Add

    cmp al, 'S'
    je do_Sub
    cmp al, 's'
    je do_Sub

    cmp al, 'M'
    je do_Mul
```

```
    cmp al, 'm'
    je do_Mul

    cmp al, 'D'
    je do_Div
    cmp al, 'd'
    je do_Div

    ret

do_Add:
    printf resultSum, resultSum_Len
    mov eax, [Value1]
    add eax, [Value2]
    call print_Int
    ret

do_Sub:
    printf resultDifference, resultDifference_Len
    mov eax, [Value1]
    sub eax, [Value2]
    call print_Int
    ret

do_Mul:
    printf resultProduct, resultProduct_Len
    mov eax, [Value1]
    imul eax, [Value2]
    call print_Int
    ret

do_Div:
    mov ebx, [Value2]
    cmp ebx, 0
    je division_Error

    printf resultQuotient, resultQuotient_Len
    mov eax, [Value1]
    mov ebx, [Value2]     ; reload, printf clobbered ebx
    cdq
    idiv ebx
    call print_Int
    ret
```

```
division_Error:
    printf divError, divError_Len
    ret

try_Again:
    printf tryAgain, tryAgain_Len
    scanf Choice, 10

    cmp byte [Choice], 'Y'
    je restart_program
    cmp byte [Choice], 'y'
    je restart_program

    Exit

restart_program:
    printf clearScreen, clearScreen_Len
    jmp void_main

; Convert input text into an integer
to_Int:
    xor eax, eax

.next:
    movzx ebx, byte [esi]
    cmp ebx, 0
    je .done
    cmp ebx, 10
    je .done

    cmp ebx, '0'
    jb .done
    cmp ebx, '9'
    ja .done

    imul eax, 10
    sub ebx, '0'
    add eax, ebx
    inc esi
    jmp .next

.done:
    ret
```

```
; Print the signed integer stored in EAX
print_Int:
    mov edi, Answer + 11
    mov byte [edi], 0
    mov ebx, 10

    cmp eax, 0
    jge .positive

    neg eax
    push eax
    mov byte [Answer], '-'
    pop eax
    mov edi, Answer + 10

.positive:
    cmp eax, 0
    jne .digits

    dec edi
    mov byte [edi], '0'
    jmp .print

.digits:
    xor edx, edx
    div ebx
    add dl, '0'
    dec edi
    mov [edi], dl
    test eax, eax
    jnz .digits

.print:
    mov eax, 4
    mov ebx, 1
    mov ecx, edi
    mov edx, Answer + 11
    sub edx, edi
    int 0x80

    printf newLine, newLine_Len
    ret
```